Universität Stuttgart
Institute of Parallel and Distributed Systems
Distributed Systems Department

Winter Term 2009/2010
Boris Koldehofe
Andreas Benzing

Operating Systems

Exercise Sheet 4

Uploaded:	Tuesday, December 8, 2009
Due Date:	Wednesday, December 16, 2009, 10:00
Discussion:	Friday, December 18, 2009, 11:30-13:00, Room 0.108

General remarks: The intent of the exercises is to help you get more familiar with fundamental concepts of modern operating systems as presented in the lecture. For that, a total of eight exercise sheets will be made available online in PDF format. You will be able to download each sheet one week in advance in order to work on the problems. Submission deadline is on *Wednesdays 10:00 am* before the respective group meeting.

Work on the problems with your group and hand in a single solution which will be graded for all members of the group. We will review your solution and return it to you in the group meeting. Everybody has to present at least one question in the group meeting. **If you hand in a solution but are not able to present it, no points will be given for that question! This also means that if you don't attend the group meeting you also won't get any points!**

For additional reference, consult the text books cited in the lecture, and also explore the Web, where you will find a plethora of information on operation systems and its related fields. You can find additional information on lectures and exercises at the lecture homepage. If you have any questions, you may contact us at `faq-bs@ipvs.uni-stuttgart.de`.

Questions

1. (3 points) Memory Management I

   (a) Consider a fixed partitioning scheme with equal-size partitions of 2^{16} bytes and a total main memory size of 2^{24} bytes. A process table is maintained that includes a pointer to a partition for each resident process. How many bits are required for the pointer?

   > **Solution:**
   > The total number of partitions in main memory is
   > $$\frac{2^{24}}{2^{16}} = 2^{24-16} = 2^8 \quad (= 256) \tag{1}$$
   > Thus, 8 bytes are required to address a partition.

   (b) To implement the various placement algorithms for dynamic partitioning (first-fit, best-fit, next-fit, worst-fit), a list of the free blocks of memory must be kept. For each of the four methods, first-first, best-fit, next-fit, and worst-fit, what is the average length of the search?

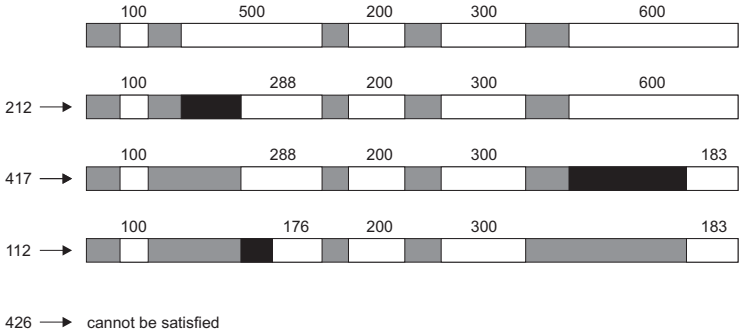
   > **Solution:**
   > First-fit: $O(n/2)$
   > Best-fit: $O(n)$ (always traverse complete list)
   > Next-fit: $O(n/2)$
   > worst-fit: $O(n)$ (always traverse complete list)

   (c) Given five memory partitions of 100 KB, 500 KB, 200 KB, 300 KB, and 600 KB (in order), how would each of the first-fit, best-fit, next-fit, and worst-fit algorithms place processes of 212 KB,

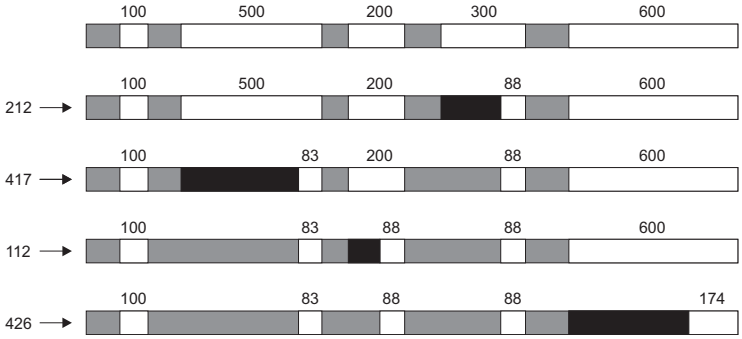
417 KB, 112 KB, and 426 KB (in order)? Which algorithm makes the most efficient use of memory?

Solution:

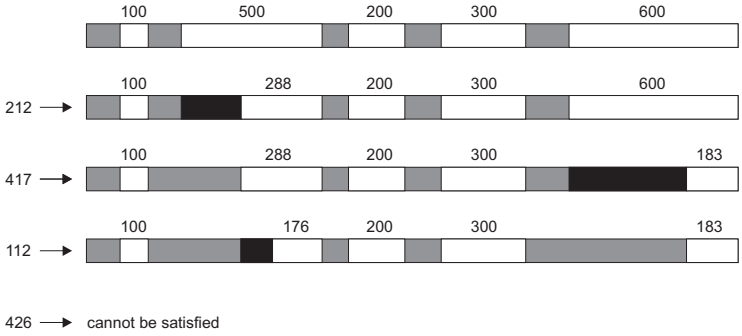
First-fit:



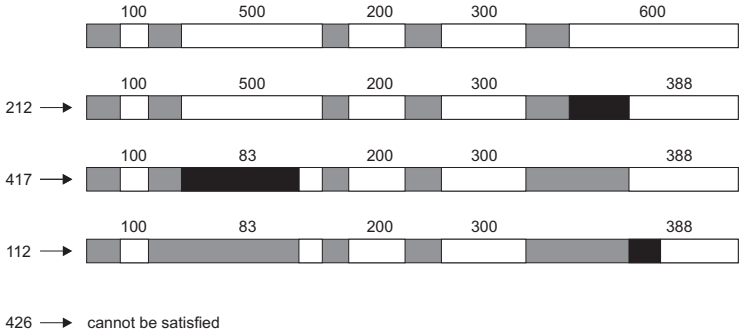
Best-fit (most efficient use of memory):



Next-fit:



Worst-fit:



(d) A 1-Mbyte block of memory is allocated using the buddy system. Show the results of the follow-

ing sequence of requests. Use a suitable representation (e.g. horizontal bars with subdivisions) to visualize the allocations.

- Request 70KB (A)
- Request 35KB (B)
- Request 80KB (C)
- Return A
- Request 60KB (C)
- Return B
- Return D
- Return C

Solution:

	1024					
request: 70	A	128		256		512
request: 35	A	B	64	256		512
request: 80	A	B	64	C	128	512
return A	128	B	64	C	128	512
request 60	128	B	D	C	128	512
return B	128	64	D	C	128	512
return D	256			C	128	512
return C	1024					

(e) Consider a simple paging system with the following parameters: 2^{32} bytes of physical memory; page size of 2^{10} bytes; 2^{16} pages of logical address space.

i. How many bits are in a logical address?

Solution:

page number: 16 bits	page offset: 10 bits

Number of bits in logical address: 26

ii. How many bytes are in a frame?

Solution:
Number of bytes in a frame: 2^{10} bytes = page size

iii. How many bits in the physical address specify the frame?

Solution:
Number of frames in physical memory: $2^{32}/2^{10} = 2^{32-10} = 2^{22}$
Number of bits specifying the frame in the physical address: 22 bits

iv. How many entries are in the page table?

Solution:
2^{16} pages $\rightarrow 2^{16}(65536)$ page table entries

v. How many bits are in each page table entry? Assume each page table entry includes a valid/invalid bit.

Solution:
22 bits to specify frame number + valid/invalid bit = 23 bits

(f) Consider a simple segmentation system that has the following segment table:

Starting Address	Length in Bytes
660	248
1752	422
222	198
996	604

For each of the following logical addresses, determine the physical address or indicate if a segment fault occurs:

i. 0, 198

Solution:
$0 \rightarrow 660 \quad + 198 \quad = 858$ (OK, $198 < 248$)
TODO (use Figure 7.12(b) in Stallings, page 321 (scan it for Powerpoint presentation)

ii. 2, 156

Solution:
$2 \rightarrow 222 \quad + 156 \quad = 378$ (OK, $156 < 198$)

iii. 1, 530

Solution:
$1 \rightarrow 1752 \quad + 530 \quad =???$ (PAGE FAULT, $530 > 422$)

iv. 3, 444

Solution:
$3 \rightarrow 996 \quad + 444 \quad = 1440$ (OK, $444 < 604$)

v. 0, 222

Solution:
$\rightarrow 660 \quad + 222 \quad = 882$ (OK, $222 < 248$) TODO

2. (4 points) Virtual Memory

(a) Suppose the page table for the process currently executing on the processor looks like the following. All numbers are decimal, everything is numbered starting from zero, and all addresses are memory byte addresses. The page size is 1024 bytes.

Virtual page number	Valid bit	Reference bit	Modify bit	Page frame number
0	1	1	0	4
1	1	1	1	7
2	0	0	0	—
3	1	0	0	2
4	0	0	0	—
5	1	0	1	0

What physical address, if any, would each of the following virtual addresses correspond to? (Do not try to handle any page faults, if any.)

i. virtual address 1052

Solution:
$1024 + 28$ maps to VPN 1 in PFN 7, $(7 * 1024 + 28 = 7196)$

ii. virtual address 2221

Solution:
$2 * 1024 + 173$ maps to VPN 2 ? PAGE FAULT

iii. virtual address 5499

Solution:
$5 * 1024 + 379$ maps to VPN 5 in PFN 0, $(0 * 1024 + 379 = 379)$

(b) A process has four page frames allocated to it (i.e. fixed allocation). (All the following numbers are decimal, and everything is numbered starting from zero). The time of the last loading of a page into each page frame, the time of last access to the page in each page frame, the virtual page number in each page frame, and the reference (R) and modified (M) bits for each page frame are as shown (the times are in clock ticks from the process start at time 0 to the event — not the number of ticks since the event to the present).

Virtual page number	Page frame	Time loaded	Time referenced	R bit	M bit
2	0	60	161	0	1
1	1	130	160	1	0
0	2	26	162	1	0
3	3	20	163	1	1

A page fault to virtual page 4 has occurred at time 164. Which page frame will have its contents replaced for each of the following memory management policies? Explain why in each case.

i. FIFO (first-in-first-out) policy

Solution:
Replace PFN 3, since time loaded (20) furthest back in time

ii. LRU (last recently used) policy

Solution:
Replace PFN 1, since time referenced (160) furthest back in time

iii. Clock policy

Solution:

VPN	Page frame	Time loaded	Time referenced	R bit	M bit
2	0	60	161	0	1
1	1	130	160	1	0
0	2	26	162	$1 \rightarrow 0(b)$	0
3	3	20	163	$1 \rightarrow 0(a)$	1

Current clock pointer: PFN 2 (one after most recently loaded page frame)
Advance pointer until R bit = 0 is encountered, on the way, clear R bits
Replace PFN 0, since first page frame having R bit = 0

iv. Optimal policy (Use this virtual page reference string: 4, 0, 0, 0, 2, 4, 2, 1, 0, 3, 2)

Solution:

VPN	Page frame	Time loaded	Time referenced	R bit	M bit
2	0	60	161	0	1
1	1	130	160	1	0
0	2	26	162	1	0
3	3	20	163	1	1

(Future) reference string: 4, 0, 0, 0, 2, 4, 2, 1, 0, 3, 2.
Replace PFN 3, since referenced furthest in the future.

(c) Given the state of memory just before the page fault in part (b), consider the following virtual page reference string:

4, 0, 0, 0, 2, 4, 2, 1, 0, 3, 2

How many page faults would occur if the working set policy with LRU were used with a window size of 4 instead of a fixed allocation? Show clearly when each page fault would occur.

Solution:

		PF				PF	PF		PF	PF	PF	
		4	0	0	0	2	4	2	1	0	3	2
pages in	3	4	0	0	0	2	4	2	1	0	3	2
memory, in	0	3	4	4	4	0	2	4	2	1	0	
LRU order	2	0	3	3			0	0	4	2	1	
	1	2								4	2	

(d) A process contains eight virtual pages on disk and is assigned a fixed allocation of four page frames in main memory. The following page trace occurs:

1, 0, 2, 2, 1, 7, 6, 7, 0, 1, 2, 0, 3, 0, 4, 5, 1, 5, 2, 4, 5, 6, 7, 6, 7, 2, 4, 2, 7, 3, 3, 2, 3

Show the successive pages residing in the four frames using the LRU replacement policy. Compute the hit ratio in main memory. Assume that the frames are initially empty.

Solution:

page trace

1	0	2	2	1	7	6	7	0	1	2	0	3	0	4	5	1	5	2	4	5	6	7	6	7	2	4	2	7	3	3	2	3
1	1	1	1	1	1	1	1	1	1	1	1	1	1	4	4	4	4	4	4	4	4	4	4	4	2	2	2	2	2	2	2	2
-	0	0	0	0	0	6	6	6	6	2	2	2	2	2	5	5	5	5	5	5	5	5	5	5	5	4	4	4	4	4	4	4
-	-	2	2	2	2	2	2	0	0	0	0	0	0	0	0	0	0	2	2	2	2	7	7	7	7	7	7	7	7	7	7	7
-	-	-	-	-	7	7	7	7	7	7	7	3	3	3	3	1	1	1	1	1	6	6	6	6	6	6	6	3	3	3	3
F	F	F			F	F		F		F		F		F	F	F		F			F	F				F	F			F		

Number of page faults: 17 (of 33 accesses)

Hit ratio: 16/33

(e) Repeat part (d) for the FIFO replacement policy.

Solution:

page trace

1	0	2	2	1	7	6	7	0	1	2	0	3	0	4	5	1	5	2	4	5	6	7	6	7	2	4	2	7	3	3	2	3
1	1	1	1	1	1	6	6	6	6	6	6	6	6	4	4	4	4	4	4	4	6	6	6	6	6	6	6	6	6	6	2	2
-	0	0	0	0	0	0	0	0	1	1	1	1	1	1	5	5	5	5	5	5	5	7	7	7	7	7	7	7	7	7	7	7
-	-	2	2	2	2	2	2	2	2	2	0	0	0	0	0	1	1	1	1	1	1	1	1	1	1	4	4	4	4	4	4	4
-	-	-	-	-	7	7	7	7	7	7	7	3	3	3	3	3	3	2	2	2	2	2	2	2	2	2	2	2	3	3	3	3
F	F	F			F	F			F		F	F		F	F	F		F			F	F					F		F		F	

Number of page faults: 17 (of 33 accesses)

Hit ratio: 16/33

(f) Assuming a page size of 4 Kbytes and that a page table entry takes 4 bytes, how many levels of page tables would be required to map a 64-bit address space if the top-level page table fits into a single page?

Solution:

Given:

- Page size: 4 KBytes = 2^{12} bytes (1 KByte = 1024 byte).

- Page table entry: 4 bytes (32 bit).
- Address space: 64-bit addresses.
- Top-level page table fits into a single page.

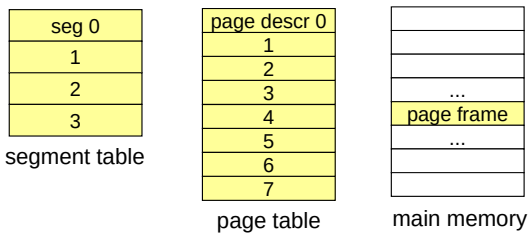
One page table contains 4096 bytes
→ 1024 page table entries that point to 2^{10} pages
→ addressing a total of $2^{10} \cdot 2^{12} = 2^{22}$ bytes (1 level)
Add another level of page tables,
→ each page table pointing to 2^{10} page tables
→ addressing a total of $2^{10} \cdot 2^{22}$ bytes $= 2^{32}$ bytes (2 levels)
Continue with additional levels:
→ total of 2^{42} bytes (3 levels)
→ total of 2^{52} bytes (4 levels)
→ total of 2^{62} bytes (5 levels)
→ Additional 6th level required, but only 2 bits (instead of 10)
Length of the virtual address:
→ offset within a frame: page size = frame size → 12 bits
→ page number: 2^{10} pages → 10 bits
→ Total: $6 \cdot 10$ bits $+12$ bits $= 72$ bits
→ However: only 2 bits required on 6th level → saves 8 bits
→ Thus: 72 bits -8 bits $= 64$ bit virtual address

(g) Consider a page referencing string for a process with a working set of M frames, initially all empty. The page reference string is of length P with N distinct page numbers in it. For each of the replacement strategies (FIFO, LRU, etc.), what is a lower and what is an upper bound on the number of page faults?

Solution:
Consider FIFO:
reference string: 1 2 3 3 3 3 3 3 ... $(N = 3, P > 3)$
→ N page faults in the beginning, then no more page faults
→ lower bound $= N$
reference string: 1 2 3 1 2 3 1 2 3 ...
if $M \leq N$ → page fault with every reference
→ upper bound $= P$
Same results for LRU and others

(h) Assume that a task is divided into four equal-sized segments and that the system builds an eight-entry page descriptor table for each segment. Thus, the system has a combination of segmentation and paging. Assume also that the page size is 2 Kbytes.

Solution:



i. What is the maximum size of each segment?

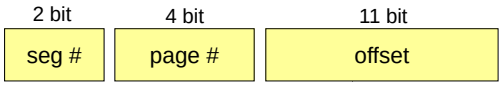
Solution:
Maximum segment size: $8 \cdot 2$ Kbytes = 16 Kbytes

ii. What is the maximum logical address space for the task?

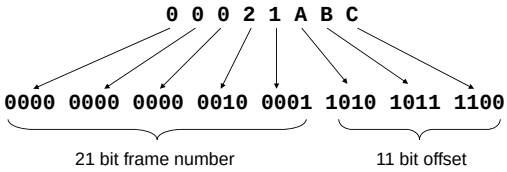
Solution:
Maximum logical address space: $4 \cdot 16$ Kbytes = 64 Kbytes

iii. Assume that an element in physical location 00021ABC (hex) is accessed by this task. What is the format of the logical address that the task generates for it? What is the maximum physical address space for the system?

Solution:
Logical address format:



Physical location:



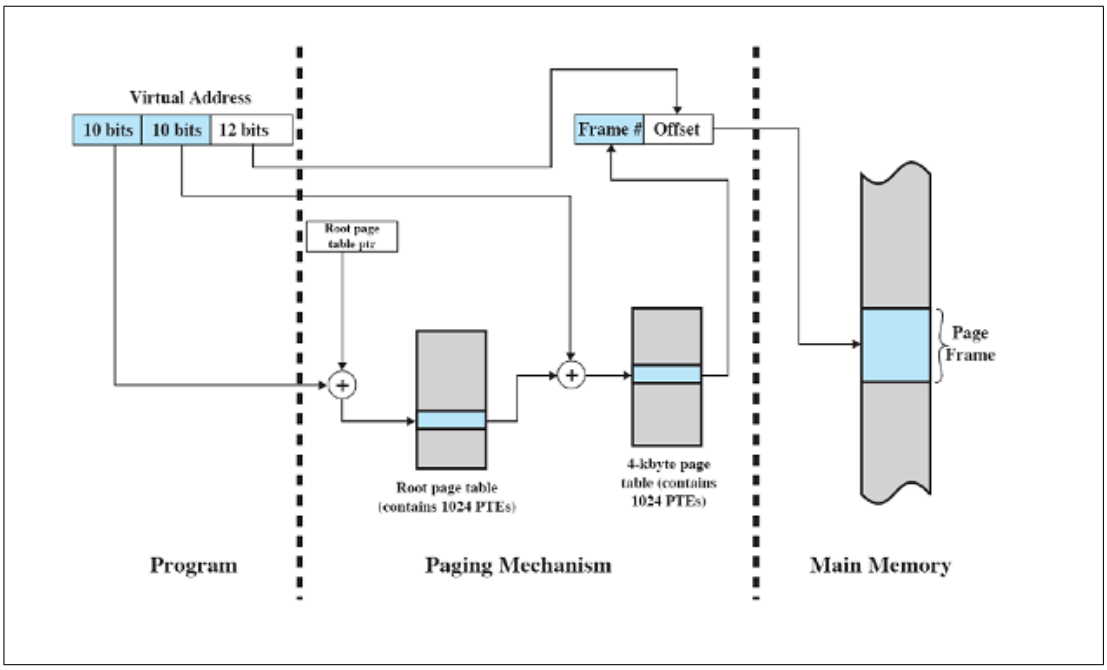
maximum physical address space: 2^{32} = 4 Gbyte

3. (3 points) Memory Management II
(Exam Questions: Part (a) and (b) from summer 2007, part (c) from summer 2008)

Consider a 32-bit logical address space, a two level paging scheme, page table entries that have a size of 4 bytes, and 4 Kbytes pages. Moreover assume byte level addressing.

(a) Sketch in a picture how the address translation of the two-level paging scheme works. Show how the paging mechanism translates a virtual address specified by a program to a physical address. Determine also all necessary values needed for the address translation (i.e. number of entries in a page table, bits assigned to the offset, bits needed to determine a frame) and explain how they can be derived.

Solution:



(b) Consider a translation lookaside buffer (TLB) with a capacity of four entries is part of the system architecture. For simplicity, assume a one-to-one mapping between logical addresses and physical addresses, i.e., logical address k is mapped to the same physical address k. Describe on the given sequence of memory accesses (in hexadecimal notation) below the functionality of the TLB and explain how it can improve the performance of a computer system.
Memory Accesses:

- 1011AFAA
- 1011AFAB
- 1011B000
- 1011AFAA

Solution:
The TLB provides a cache for pages. In the scheme a process that accesses A1 will receive a pointer to the frame A1. The physical memory location can be obtained directly from the offset. Assuming the access to the TLB is more efficient than access to main memory and memory access shows good locality we gain performance by accessing the TLB. In particular, in a two level hierarchy we have one access to the TLB compared to two memory accesses in the two-level page hierarchy to determine a frame.
In the particular sequence:

- 1011AFAA: 10111A is not contained in the TLB so we have a cache miss and need to go through the two level hierarchy: Two memory accesses are necessary + 1 finding the location
- 1011AFAB: 10111A is part of the TLB – TLB returns address 1011A we can use the offset FAB to access the memory location
- 1011B000: 1011B is not part of theTLB – same procedure
- 1011AFAA: 1011A is part of the TLB – directly access the memory location

(c) Consider a demand-paging system with the following time-measured utilizations:

- CPU: 20%
- Paging disk: 97.7%
- Other I/O devices: 5%

What effects will each of the following changes to the system have on CPU and paging disk utilizations? Explain your answer.

i. Installing a faster CPU

> **Solution:**
> The system obviously is spending most of its time paging, indicating over-allocation of memory and under-utilization of CPU. Therefore, installing a faster CPU will not help.

ii. Increasing the degree of multiprogramming

> **Solution:**
> By increasing multiprogramming the resident processes will page fault even more. Therefore, CPU utilization would not improve.

iii. Decrease the degree of multiprogramming

> **Solution:**
> If the level of multiprogramming is reduced resident processes would page fault less frequently and the CPU utilization would improve.

iv. Install more main memory

> **Solution:**
> Likely to improve CPU utilization as more pages can remain resident and not require paging to or from the disks.

v. Add a paging cache

> **Solution:**
> Pages could be pre-fetched and kept in the cache. The CPU will get more data faster so it will be more in use.

vi. Decrease the page size

> **Solution:**
> Decreasing the page size will result in more page faults if the data is being accessed sequentially. Therefore, CPU utilization would be decreased.